



CSE 4111

Compiler Design

Sakurah Ismail

Lecturer

Department of Computer Science & Engineering

North Bengal International University

Language & Grammar

- In Compiler design, language is a collection of all possible strings for a given grammar set.
- Grammar in theory of computation is a finite set of formal rules that are generating syntactically correct sentences.
- Context-free grammars are named as such because **any of the production rules in the grammar can be applied regardless of context**—it does not depend on any other symbols that may or may not be around a given symbol that is having a rule applied to it.

Regular Expressions

- **Regular Expression (Regex)** is a **pattern** used to describe a set of strings.
- The languages accepted by some regular expression are referred to as Regular languages.
- A regular expression can also be described as a sequence of symbols that defines a pattern for a set of strings.
- **For instance:**
 - In a regular expression, x^* means zero or more occurrence of x . It can generate $\{\epsilon, x, xx, xxx, xxxx, \dots\}$
 - In a regular expression, x^+ means one or more occurrence of x . It can generate $\{x, xx, xxx, xxxx, \dots\}$

Operations on Regular Language

- The various operations on regular language are:

1. Union ($\cup$)

Combines two languages.

Strings that are in L_1 **OR** L_2

$$L = L_1 \cup L_2 = \{ w \mid w \in L_1 \text{ or } w \in L_2 \}$$

Example:

- $L_1 = \{a, b\}$
- $L_2 = \{b, c\}$
- $L_1 \cup L_2 = \{a, b, c\}$

Concatenation ($\cdot$)

Join strings of two languages.

First take a string from L_1 , then from L_2

$$L = L_1 \cdot L_2 = \{ xy \mid x \in L_1, y \in L_2 \}$$

Example:

- $L_1 = \{a, b\}$
- $L_2 = \{c\}$
- $L_1 L_2 = \{ac, bc\}$

Kleene Star (*)

Repetition of strings **zero or more times**

Example:

$$L = \{a\}$$

$$L^* = \{\epsilon, a, aa, aaa, \dots\}$$

Kleene Plus (+)

Repetition **one or more times**

- $L = \{a\}$
- $L^+ = \{a, aa, aaa, \dots\}$

Intersection ($\cap$)

Common strings of two languages

$$L = L_1 \cap L_2 = \{ w \mid w \in L_1 \text{ and } w \in L_2 \}$$

Example:

- $L_1 = \{a, b\}$
- $L_2 = \{b, c\}$
- $L_1 \cap L_2 = \{b\}$

Examples of Regular Expressions

Example 1:

- Write the regular expression for the language accepting all combinations of a's, over the set $\Sigma = \{a\}$

Solution:

- All combinations of a's means a may be zero, single, double and so on. If a is appearing zero times, that means a null string. That is we expect the set of $\{\epsilon, a, aa, aaa, \dots\}$. So we give a regular expression for this as:

$$R = a^*$$

That is Kleen closure of a.

Examples of Regular Expressions (Cont.)

Example 2:

- Write the regular expression for the language accepting all combinations of a's except the null string, over the set $\Sigma = \{a\}$

Solution:

The regular expression has to be built for the language

$$L = \{a, aa, aaa, \dots\}$$

This set indicates that there is no null string. So we can denote regular expression as:

$$R = a^+$$

Examples of Regular Expressions (Cont.)

Example 3:

- Write the regular expression for the language accepting all the string containing any number of a's and b's.

Solution:

- The regular expression will be:

$$\text{r.e.} = (a + b)^*$$

This will give the set as $L = \{\epsilon, a, aa, b, bb, ab, ba, aba, bab, \dots\}$, any combination of a and b.

The $(a + b)^*$ shows any combination with a and b even a null string

Examples of Regular Expressions (Cont.)

Example 4:

- Write the regular expression for the language accepting all the string which are starting with 1 and ending with 0, over $\Sigma = \{0, 1\}$.

Solution:

- In a regular expression, the first symbol should be 1, and the last symbol should be 0. The r.e. is as follows:

$$\text{R.E.} = 1 (0+1)^* 0$$

Assignments

i.

Write the regular expression for the language starting with a but not having consecutive b's.

ii.

Write the regular expression for the language accepting all the string in which any number of a's is followed by any number of b's is followed by any number of c's.

iii.

Write the regular expression for the language over $\Sigma = \{0\}$ having even length of the string.



Context Free Grammar (CFG) (Cont.,)

- A CFG is a set of rules/productions used to generate patterns of strings.

- If a language like, $L = \{ \text{strings containing 0 and 1} \}$
 $= \{01, 011, 0011, 0111, 000111, \dots\}$

But we want to generate the pattern like

01,
0011,
000111

Then we have to apply a rule on this language, $L = \{ 0^n 1^n \mid n \geq 1 \}$

The grammar will be

$A \rightarrow 01 \mid 0A1$ (A derives)

i.e., $A \rightarrow 01$ or $A \rightarrow 0A1$

$$n=1; 0^1 1^1 = 01$$

$$n=2; 0^2 1^2 = 0011$$

$$n=3; 0^3 1^3 = 000111$$

The derivation is

$A \rightarrow 0A1$

$\rightarrow 00A11$

$\rightarrow 000111$

Our target pattern is 000111

Ambiguity in Grammar

A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string. If the grammar is not ambiguous, then it is called unambiguous.

If the grammar has ambiguity, then it is not good for compiler construction. No method can automatically detect and remove the ambiguity, but we can remove ambiguity by re-writing the whole grammar without ambiguity.

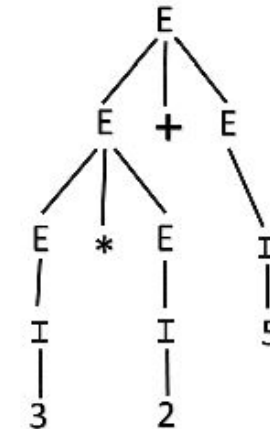
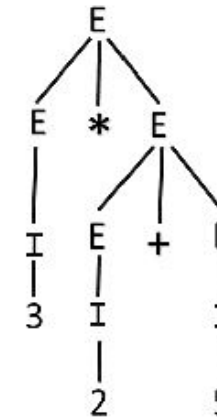
Q: What is Ambiguous grammar?

Examples of Ambiguous Grammar

- Example 1:
- Let us consider a grammar G with the production rule
- $E \rightarrow I$
- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow (E)$
- $I \rightarrow \varepsilon \mid 0 \mid 1 \mid 2 \mid \dots \mid 9$

For the string "3 * 2 + 5", the above grammar

Can generate two parse trees



Q: Check whether the grammar is ambiguous or not.

Examples of Ambiguous Grammar (Cont.)

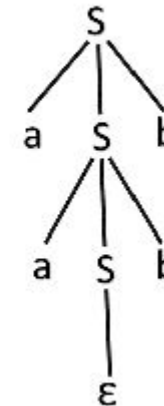
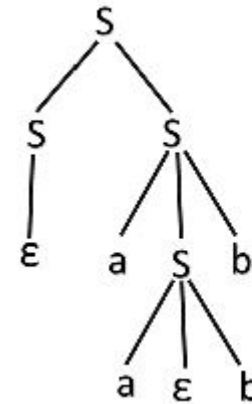
Example 2:

- Check whether the given grammar G is ambiguous or not.

$S \rightarrow aSb \mid SS$

$S \rightarrow \epsilon$

For the string "aabb" the above grammar can generate two parse trees



Since there are two parse trees for a single string "aabb", the grammar G is ambiguous.

Q: Check whether the grammar is ambiguous or not.

Unambiguous Grammar

A grammar can be unambiguous if the grammar does not contain ambiguity that means if it does not contain more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string.

Q: What is unambiguous grammar?

Ambiguous to Unambiguous Grammar

Example 1:

Show that the given grammar is ambiguous. Also, find an equivalent unambiguous grammar.

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Solution:

Let us derive the string "**id + id * id**"

- As there are two different parse tree for deriving the same string, the given grammar is ambiguous.
- Unambiguous grammar will be:

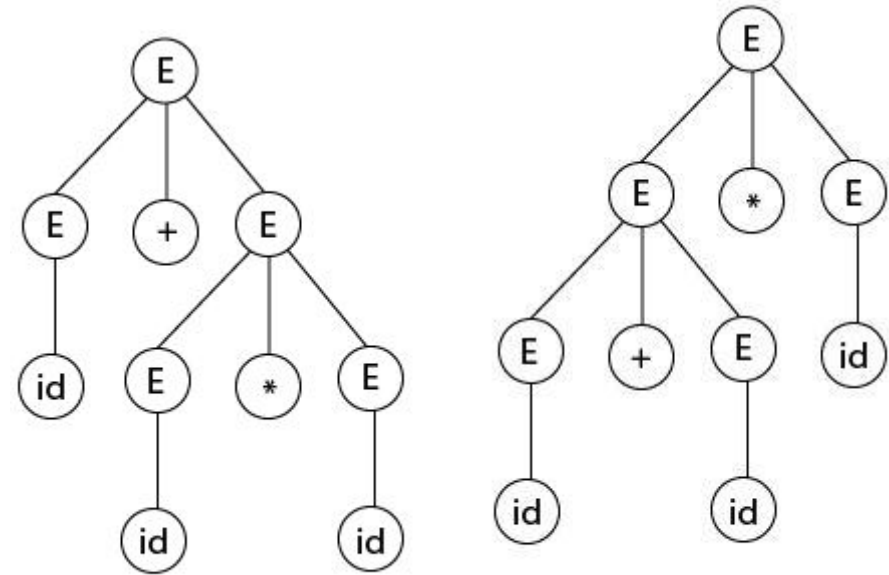
$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow id$



Parse tree 1

Parse tree 2

Parse Tree

Rules to Draw a Parse Tree :

- All leaf nodes need to be terminals.
- All interior nodes need to be non-terminals.
- In-order traversal gives original input string.

Example-1:

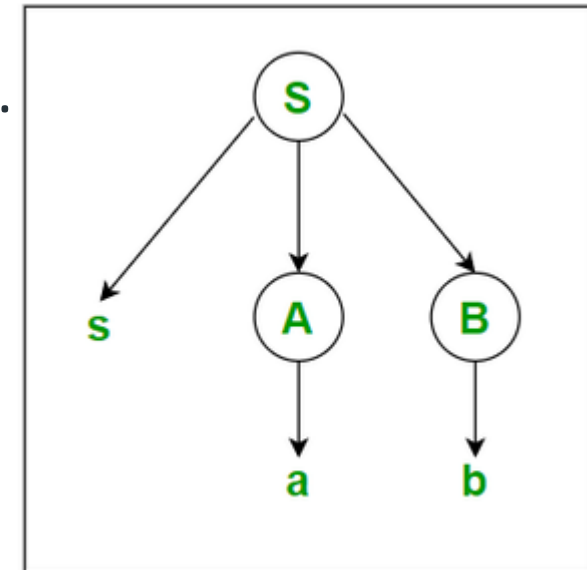
Let us take an example of a Grammar (Production Rules).

$S \rightarrow sAB$

$A \rightarrow a$

$B \rightarrow b$

The input string is “sab”, then the Parse Tree is :



How does parse tree work?

- Parse tree is the hierarchical representation of terminals or non-terminals.
- These symbols (terminals or non-terminals) represent the derivation of the grammar to yield input strings.
- The starting symbol of the grammar must be used as the root of the Parse Tree.
- Leaves of parse tree represent terminals.
- Each interior node represents productions of grammar.

Parse Tree Examples

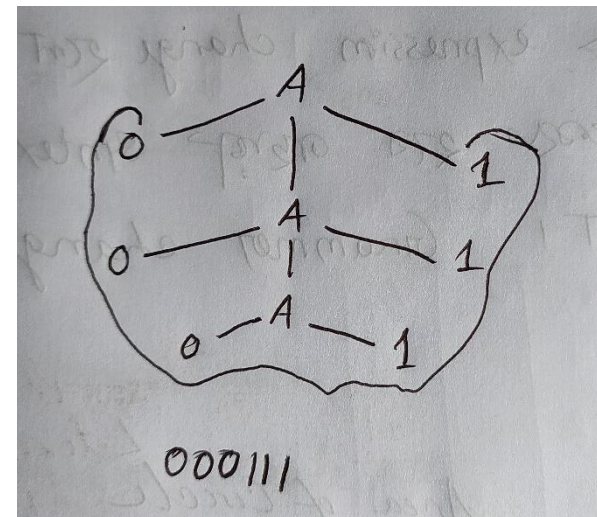
➤ Parse Tree:

A **parse tree** or **parsing tree** or **derivation tree** or **concrete syntax tree** is an ordered, rooted tree that represents the syntactic structure of a string.

As we know our target pattern is 000111. By using parse tree, we can get our target pattern.

$A \rightarrow 01 \mid 0A1$ (A derives)
i.e., $A \rightarrow 01$ or $A \rightarrow 0A1$

Q: What is parse tree?



Parse Tree Examples

Example-2:

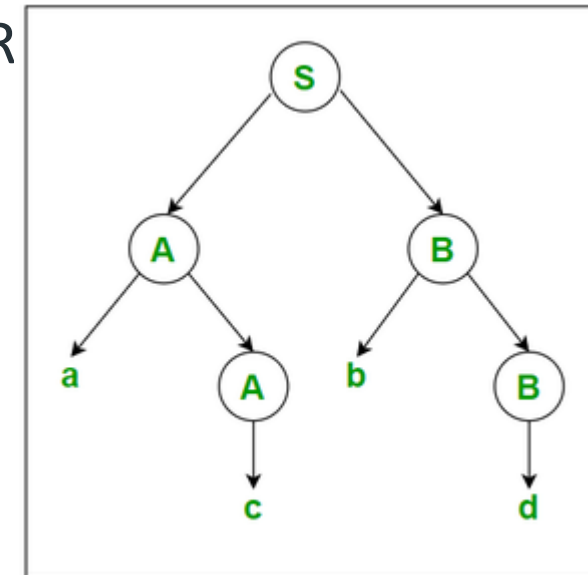
Let us take another example of a Grammar (Production R

$S \rightarrow AB$

$A \rightarrow c/aA$

$B \rightarrow d/bB$

The input string is “acbd”, then the Parse Tree is :



Parse Tree & Syntax Tree

- Let the expression is

$id_1 = id_2 + id_3 * 60;$

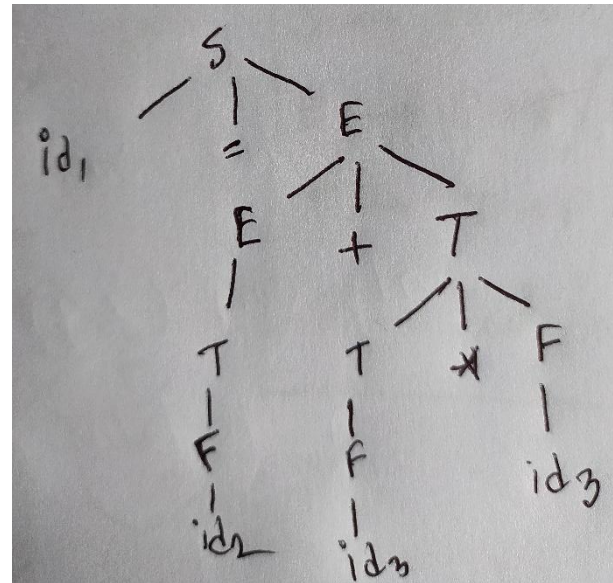
The grammar of this expression is

$S \rightarrow id = E$

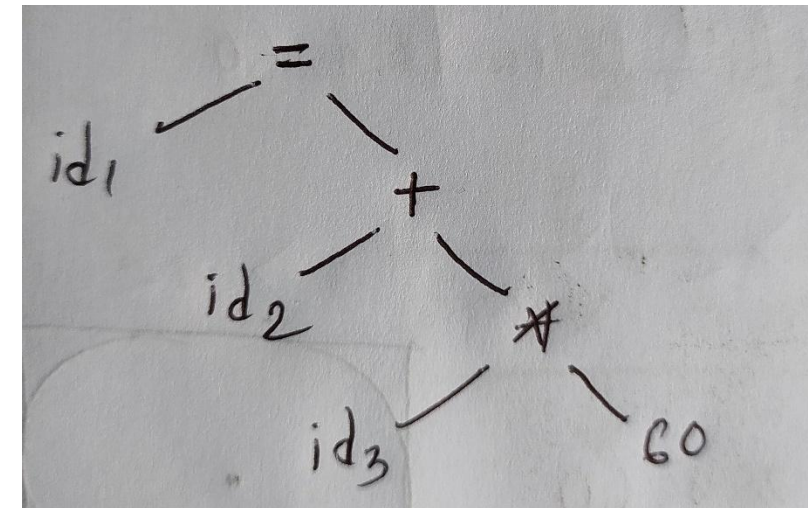
$E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow id / num$



Parse
tree



Syntax
tree

Difference between parse tree & syntax tree



Syntax Tree	Parse Tree
interior nodes are "operators", leaves are operands	interior nodes are non-terminals, leaves are terminals
when representing a program in a tree structure usually use a syntax tree	rarely constructed as a data structure
Represents the abstract syntax of a program (the semantics)	Represents the concrete syntax of a program
Contain only meaningful information.	Contain unusable information also.
Syntax tree examples Grammar: $E \rightarrow E * E \mid E + E \mid id$ Program: $a + b * c$ Syntax tree <pre> graph TD A["+"] --- B[id] A --- C["*"] C --- D[id] C --- E[id] </pre>	Parse tree examples Grammar: $E \rightarrow E * E \mid E + E \mid id$ Program: $a + b * c$ Parse tree <pre> graph TD A[E] --- B[E] A --- C["+"] A --- D[E] B --- E[id] D --- F[E] D --- G["*"] D --- H[E] F --- I[id] H --- J[id] </pre>



Thank You